

Break Check

Capstone Proposal — EECE4791 Team 1

Neil Singh, Siwon Kang, Ajey Sasimugunthan, Noah Carlson, Klaidas Wikar, Keaton
Zelazny

Advisor: Dr. Fatema Nafa

Department of Electrical and Computer Engineering
Northeastern University
June 21, 2026

Abstract

This proposal discusses creating an interactive heads-up display that gives a user an overview of their vehicle status in real-time in order to give the driver the knowledge to prevent potential damages.

This will be accomplished by decoding the vehicle's CAN bus codes, processing the data through a model that can detect anomalies, and displaying the overview in a readable format to the user. This will leverage key technologies such as UART-v13 CAN frame decoding, an LSTM Autoencoder hosted via TensorFlow Lite, and a PyGame display.

We expect the outcome to be a device with a sleek interface that accurately detects system anomalies/faults. This should positively impact those who want to prevent unexpected breakdowns and maintenance fees. We will test the accuracy of our by comparing it against a benchmark simple "Rolling Z-Score" model.

Contents

Abstract	1
1 Introduction	3
2 Objectives	3
2.1 Project Objectives	3
2.2 Learning Objectives	3
3 Problem Specification	3
3.1 Requirements and Success Criteria	4
4 Related Works	4
5 System Overview and Architecture	5
6 Methodology	5
7 Design Strategies and Analysis	6
7.1 Hardware / Compute	6
7.2 Software / Model Design	7
7.2.1 Data Pipeline	7
7.2.2 Model Choice and Trade-Offs	7
7.2.3 Training Data Strategy	8
7.2.4 Application / UI	9
8 Tools List	9
9 Testing and Verification Plan	10
10 Results and Evaluation	11
11 Cost Analysis	11
12 Team Contribution	11
13 Proposed Timeline	12

14 Next Semester Plan	13
15 Conclusion	14
References	15
A Appendix: Additional Material	16

1 Introduction

Since the dawn of the automotive era, everyday drivers have struggled with the cost of unforeseen damages to their vehicles at the benefit of mechanics. Right now, a vague problem may arise in a driver's vehicle, of which they have no way to understand or prevent without talking to a mechanic. Our goal in this project is to flip that hierarchy over and give power back to the consumer.

We will accomplish this by creating a device that gives all of the information a driver will need to know about their vehicle in real-time, right on their dash. Prior tools to address this problem fall short due to obscurity, lack of clean interface, and poor issue framing. We aim to learn from these downfalls by making a product that is easy to use, presents useful, understandable information, and is appealing to the general consumer, not just car enthusiasts.

2 Objectives

The goal is to help give drivers real time data on their vehicle's health and predict issues ahead of time through an affordable product that connects to the OBD port of the car.

2.1 Project Objectives

- The primary capability of the product is to be able to predict issues in the car before they occur.
- The secondary capability of the product is to provide a clean display that shows all the vehicle health metrics in a way that anyone can easily understand.
- The target is to be able to create a product that can predict issues ahead of time with high accuracy and to be able to do so in such a way that allows drivers to interact with the interface easily.

2.2 Learning Objectives

- One skill would be data acquisition and processing as we will interacting with the car's OBD-II port to convert raw vehicle information into words that everyone can understand.
- A second skill we will be learning is machine learning and AI. A prediction model is something we will have to work on to predict issues for our project.
- Programming is a skill that we will be building in order to program the display and be able to ensure seamless Bluetooth connection.
- Another skill that will be gained is wireless communication protocols as we will learn to debug the Bluetooth module with the product and display module.
- The fifth learning objective is systems integration as our group will learn how to simultaneously merge hardware and software systems with one another.

3 Problem Specification

Project Goal. Our group is building a device that can be plugged into any car's OBD-II port and provide real time insight into their car's health and let them know about potential issues before they occur.

Problem Statement. The issue is that most drivers have no real insight into their car’s health. When a warning light comes on the dashboard, most drivers are left confused and it usually requires an expensive trip to the mechanic. There are products on Amazon that can read faulty codes but are purely reactive. The warning light is a reaction meaning damage has already been done to the car. Meaning, most drivers pay more than they should to figure out their car’s health and then having to fix those issues.

Desired Result.

- **Input:** Raw data from the car’s ECU via the OBD-II port will be obtained using a Bluetooth device. This includes details such as miles per hour, trouble codes, and batter voltage.
- **Output:** A display will show live readings from the car’s ECU on the vehicle’s health in a way that anyone can understand.

3.1 Requirements and Success Criteria

Table 1: Functional and non-functional requirements.

ID	Requirement	Target / Acceptance criterion
R1	display real time vehicle data to the user	readings come up on display in no less than one second
R2	trouble codes translated into plain English	100 percent of fault codes translated into plain English
R3	maintain stable Bluetooth between OBD port and display	starts up in 10 seconds and no issues over a 30 minute span
R4	predict potential issues before they occur using a ML model	prediction model achieves ≥ 90 percent accuracy

4 Related Works

Our topic builds on existing consumer products which can plug into the car’s OBD-II port and provide trouble codes to the consumer. However, it can be hard to understand what they are trying to convey and do not predict issues ahead of time. There is academic ML research that exists but none that can be applied to an actual car. This gap is where our product lives.

- **Commercial Tools** (BlueDriver, FIXD) [1]. These tools can connect to a car’s OBD-II port and read diagnostic codes from the car and tell the user over an app using their smartphone.
- **ML Academic Research** [2]. Researchers have created ML models that can predict issues before a warning light comes on with over ninety percent accuracy.
- **ScanGauge II** [3]. This is a tool that acts like a digital gauge cluster and provides information on speed but nothing about diagnostic codes.

Connections to Our Project

- These products show that there is a demand for a product like ours and we'd be covering a deficiency as they cannot predict issues ahead of time.
- This research shows us that it is possible to create a prediction model that can predict issues ahead of time and put in a package that consumers can use.
- The product confirms our decision to make our own standalone display and have the capability to predict issues so that consumers can save real time and money.

5 System Overview and Architecture

Our proposed system, Break Check, will consist of 4 main components. As seen in Figure 1, the first component is a UART-v13 board that takes data from a user's vehicle (OBD-II port). Then that data will be transmitted to another device made up of a Raspberry Pi 4, used to perform predictive analysis of wear patterns. The results of that analysis are then transmitted to a TI DLP Pico, which projects the data onto a driver's heads-up display for the driver to see. This includes live vehicle gauges, plain-English trouble codes, and real-time anomaly alerts generated by the predictive model.

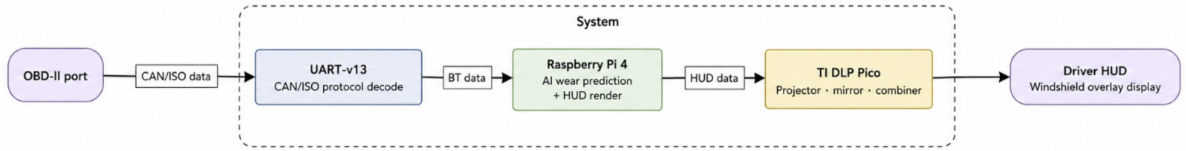


Figure 1: System block diagram: inputs, processing, and outputs.

6 Methodology

The end-to-end workflow as shown in figure 2 consists of using data from multiple sources, cleaning and formatting that data, and preparing the data for training and validating the model. The numbered list explains each stage in more detail.

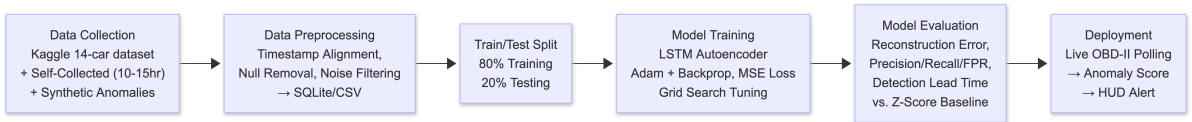


Figure 2: Methodology pipeline: data → preprocessing → train/test split → model training → evaluation → deployment.

1. **Data collection.** Kaggle OBD-II Dataset; 14 drivers/14 vehicles, 7s sampling interval; PID readings (RPM, speed, coolant temp, etc.) | Self-collected baseline data; 10-15 hours of driving; PID readings | Synthetic anomaly injections; 20 of all data; PID readings
2. **Data preprocessing.** There will likely be some timestamp alignment, removal of null/missing PID readings, and noise filtering on raw sensor values. The PID values will be what we want formatted via timestamped readings stored in SQLite and exported to CSV.

3. **Train/test split.** Take each of the 3 data sources and split the data 80% for training and 20% for testing
4. **Model training.** LSTM Autoencoder. Weights are tuned by backpropagation using gradient descent (Adam optimizer) to minimize reconstruction error. Parameters are tuned by grid search over validation performance.
5. **Model evaluation.** Metrics are reconstruction error, precision, recall, false-positive rate, and detection lead time. Validation method used will be to evaluate reconstruction error on healthy validation data to set a normal-range baseline, then test detection performance on the synthetic anomaly test set, benchmarked against the rolling z-score baseline
6. **Deployment / application.** Live sensor data will be automatically polled from the vehicle via the OBD-II port and used by the model. The output would be a score that flags a potential issue.

7 Design Strategies and Analysis

Our capstone project involves multiple aspects of hardware and software design choices. In the planning stages of the Break Check system, it was agreed upon that the most important system to devote the most time to was the development of the predictive AI system, as it involves the testing of multiple types of models and data methodology. Due to the nature of our time frame, we decided the best decision was to purchase off-the-shelf boards as opposed to manufacturing our own, to better balance the time spent between hardware and software configuration.

7.1 Hardware / Compute

The hardware components of the project need to follow the outline given previously in the system overview of real ECU data decoding, blue-tooth connection to HUD, AI framework, as well as digital image projection. The OBDII-UART-v1.3 acts as the communication bridge between the vehicle's OBD-II diagnostic port and Break Check's main computing system. converts OBD-II data into a simpler UART serial format that can be sent to the subsequent board via blue=tooth connection, essentially wirelessly connecting the vehicle ECU to the system. It uses the ELM327-style command set, meaning the system can send commands to request vehicle information such as engine RPM, vehicle speed, coolant temperature, throttle position, and diagnostic trouble codes. The Raspberry Pi 4 serves as the main computing unit for the car error decoder, handling both graphics processing and predictive AI functions. After the OBDII-UART-v1.3 converts vehicle diagnostic data into serial information, the Raspberry Pi 4 receives and processes that data using its onboard processor and memory. Its quad-core CPU allows it to organize live vehicle data, decode diagnostic trouble codes, and run software that determines the severity of each issue. For the graphical side of the system, the Raspberry Pi 4 formats the processed data into a clean driver interface, including warning messages, error code descriptions, live sensor readings, and visual indicators. This output can then be sent to a display or HUD projection system. The Raspberry Pi 4 is also responsible for running the predictive AI portion of the project. Using stored diagnostic data, live OBD-II readings, and patterns from previous vehicle faults, the AI model can estimate potential wear or upcoming issues before they become serious failures. The DLP Pico is used as the image projection component of the car error decoder's HUD system. After the Raspberry Pi 4 processes the vehicle data and creates the graphical display, the visual output is sent to the DLP Pico projector. The projector uses digital light processing technology to create a bright, compact image that can be reflected onto the windshield or a transparent combiner surface. This allows important vehicle information, such as diagnostic trouble codes, warning messages, live sensor readings, and predictive AI alerts,

to appear in the driver's line of sight without requiring the driver to look down at a separate screen. This setup allows the Break Check system to present real-time vehicle diagnostics in a more convenient and futuristic format.

7.2 Software / Model Design

7.2.1 Data Pipeline

The data pipeline will occur in four stages: CAN frame acquisition, decoding, code storage, and information distribution. At the vehicle end, the Engine Control Unit (ECU) broadcasts sensor telemetry over the Controller Area Network (CAN) bus. An OBD-II connector will connect to this bus to transmit CAN frames.

Because the Raspberry Pi cannot interpret the CAN frames directly, a UART-v13 translator board will be connected between the OBD-II port and the Raspberry Pi. The translator with its UART transmitter re-encodes the decoded parameter data as standard asynchronous serial bytes. The Pi receives these bytes over its UART interface [1].

On the Raspberry Pi, a dedicated data-service process will run a continuous polling loop using the python-obd library. Each poll cycle will request a configurable set of Parameter IDs (PIDs) such as: engine RPM, vehicle speed, coolant temperature, throttle position, intake-air temperature, battery voltage, etc., and writes every timestamped reading to a local SQLite database. At the same time, the process will publish the latest values to both the ML-service process and the display process.

A JSON configuration file will govern the entire pipeline. It will specify which PIDs to monitor, the polling interval, alert thresholds, the anomaly-detection model path, and the HUD layout parameters. Centralizing these settings allows rapid reconfiguration during testing or demonstration without recompiling or redeploying code.

7.2.2 Model Choice and Trade-Offs

The predictive component of this system will aim to detect emerging mechanical faults before they trigger a Diagnostic Trouble Code (DTC). Four candidate approaches were evaluated; each makes different assumptions about the available training data and the temporal structure of the problem.

- Rolling Z-Score (Benchmark)

This simple approach maintains a running mean μ_i and standard deviation σ_i for each monitored sensor i over a trailing window. A reading x_i is flagged when its z-score exceeds a threshold, τ [2]:

$$z_i = \frac{x_i - \mu_i}{\sigma_i} > \tau \quad (1)$$

This method is computationally trivial and easy to implement, making it a useful baseline against more sophisticated models. Its greatest limitation is that it evaluates each reading independently, so a slowly drifting sensor continually shifts the running statistics and may never breach the threshold until the fault is well advanced.

- Isolation Forest

Isolation Forest is an unsupervised algorithm that learns a profile of "normal" by randomly partitioning the feature space. Normal observations, which cluster densely, require many partitions to isolate; anomalous observations, which sit in sparse regions, are isolated quickly. An anomaly score is derived from the average path length across an ensemble of random trees [3].

Like the rolling z-score, Isolation Forest operates on individual snapshots rather than temporal sequences. It is more expressive, but it still cannot detect a gradual trend that remains within the normal envelope at every individual time step.

- Random Forest Classifier

A Random Forest classifier could, in principle, learn a mapping from sensor snapshots to fault categories [4] ("normal", "pre-failure type A," etc.). It is accurate, interpretable, and lightweight at inference time. However, it requires labeled training data: examples explicitly tagged with the actual maintenance revelation. Obtaining genuine pre-failure labels from real vehicles is impractical within the scope of this project, as it would demand either access to a fleet with documented maintenance histories or deliberate induction of mechanical faults. This data requirement makes the supervised approach a poorer fit for our constraints.

- LSTM Autoencoder (Selected Approach)

A Long Short-Term Memory (LSTM) autoencoder addresses the limitations identified above. The architecture consists of an encoder LSTM that compresses a fixed-length window of multivariate sensor readings into a low-dimensional latent vector, and a decoder LSTM that reconstructs the original window from that vector. The model is trained exclusively on *healthy* driving data, no failure labels are needed. During training, the network minimizes reconstruction error; it learns the temporal patterns that characterize normal operation (e.g., the characteristic coolant-temperature rise curve after a cold start, the correlation between RPM and throttle position during steady-state cruising).

At inference time, the reconstruction error for each new window serves as an anomaly score. A window whose patterns match the training distribution will be reconstructed accurately (low error). A window containing an emerging fault, such as a sensor drifting out of its learned envelope or an unusual oscillation developing, will reconstruct poorly (high error), triggering an alert.

The key advantages over the alternatives are:

1. **No labeled failure data required.** The model is trained only on normal driving, sidestepping the data-scarcity problem that disqualifies the Random Forest approach.
2. **Temporal awareness.** Because the LSTM processes sequences rather than snapshots, it can detect slow drifts that point-in-time methods like Isolation Forest miss.
3. **Multivariate correlation modeling.** The autoencoder jointly models all sensor channels, capturing cross-sensor relationships (e.g., coolant temperature should rise with sustained engine load).

The primary trade-off is computational cost [5]. Training will be performed offline on a workstation using TensorFlow; the trained model will then be exported to TensorFlow Lite for on-device inference on the Raspberry Pi.

7.2.3 Training Data Strategy

Because no single data source meets all requirements, the training corpus is assembled from three complementary sources:

- **Public OBD-II datasets** (e.g., from Kaggle) will provide driving-pattern diversity across multiple vehicles, drivers, and conditions [6].

- **Self-collected baseline data** will be logged from the team’s test vehicle over approximately 10–15 hours of mixed driving (city, highway, cold starts) provides a real-world anchor and serves as the primary validation set.
- **Synthetic anomaly injection** will simulate pre-fault conditions by programmatically introducing realistic deviations into the healthy data. For example, a linear coolant-temperature drift of 15% over a simulated week, or increasing RPM variance. These synthetic faults are modeled on documented failure signatures from automotive repair literature and forums.

The rolling z-score baseline will be implemented alongside the LSTM autoencoder so that the final evaluation can quantify the value added by the more complex model. If the autoencoder’s detection lead time (how far in advance it flags an emerging fault compared to the z-score) is not meaningfully greater on the synthetic test set, the simpler model would be preferred. This outcome in itself would help pave a clearer way for which model to use.

7.2.4 Application / UI

The heads-up display will be rendered by a full-screen PyGame application running on the Raspberry Pi. It will feature custom gauges of the user’s choice, a machine learning powered status overview that displays system anomalies, and any error codes deemed critical enough to display, such as a piston misfire. If the data service stalls, the display will continue to show the last known-good readings with a visual staleness indicator rather than freezing. Because the driver views the image as a reflection off the combiner glass, every frame will be horizontally mirrored before it is sent to the HDMI output.

PyGame was selected over alternatives such as Kivy or a browser-based renderer (Flask served to Chromium in kiosk mode) primarily for its low overhead. It provides direct control over per-frame drawing without the memory and CPU cost of a full GUI toolkit or web engine, which is important on the resource-constrained Pi.

8 Tools List

Raspberry Pi 4

- CPU: Quad-core ARM Cortex-A72 64-bit CPU
- RAM: 8 GB
- GPU: Broadcom VideoCore VI

OBDII-UART-v13

- Main Chip: STN1110 OBD-to-UART interpreter
- Connection: OBD-II

Texas Instruments DLP Pico

- Display: 540x960p
- Projection Element: Digital Micro-Mirroring
- Brightness: 150 Lumen
- Resolution sizes: Up to 25" display

Reflection Mirror/Display Glass

- Display: Beamsplitter Glass
- Optical System Housing: 3D Printed Housing to store mirror-display setup

Software.

- Language: Python 3 (for OBD communication, ML pipeline, data processing, and the PyGame display layer)
- Libraries: python-obd (OBD-II communication), tensorflow / tensorflow-lite (LSTM autoencoder training and inference), scikit-learn (baseline anomaly detection, data preprocessing), numpy and pandas (data wrangling and feature engineering), pygame (HUD rendering), sqlite3 (data logging, built into Python)
- Version control / environment: GitHub for version control, Raspberry Pi OS Lite as the deployment environment, and a laptop/desktop environment for model training

9 Testing and Verification Plan

The Break Check system will be tested through unit testing, integration testing, and full system-level validation. Unit tests will verify that each subsystem works independently, including OBD-II data acquisition, trouble code translation, Bluetooth communication, predictive model output, and PyGame HUD rendering. Integration testing will then confirm that data can move through the full pipeline from the OBD-II port to the Raspberry Pi, through the predictive AI model, and finally to the HUD display. System-level testing will be performed with the complete prototype connected to a vehicle or simulated OBD-II data source to confirm that the final design meets the functional requirements listed in Table 1.

Table 2: Testing and verification matrix. Each test maps directly to a requirement in Table 1.

Test	Req.	Method	Pass Criterion
T1	R1	Measure the time between OBD-II data reception and the updated value appearing on the HUD display. Test with live or simulated PID readings such as RPM, speed, coolant temperature, throttle position, and battery voltage.	Vehicle readings appear on the display in less than 1 second.
T2	R2	Inject a known set of diagnostic trouble codes into the software and compare the displayed explanation against the project diagnostic code dictionary.	100% of tested fault codes are translated into correct plain-English descriptions.
T3	R3	Run the OBD-II to Raspberry Pi Bluetooth connection continuously for 30 minutes while logging connection status, dropped packets, reconnect attempts, and data update rate.	Bluetooth starts within 10 seconds and remains stable for 30 minutes with no major communication failures.
T4	R4	Test the LSTM autoencoder using healthy driving data and synthetic anomaly data. Compare results against the rolling z-score benchmark using accuracy, precision, recall, false-positive rate, and detection lead time.	Predictive model achieves at least 90% accuracy and performs better than the rolling z-score baseline.

10 Results and Evaluation

Our evaluation focuses on three main areas: data transmission, predictive accuracy, and display reliability.

- **Data Pipeline:** We will evaluate the response time from when the car generates data at the OBD-II port to when it reaches the Raspberry Pi. Success means the data transfers almost instantly without dropping any information, ensuring the driver sees real-time updates.
- **Predictive Accuracy:** We will test our AI's ability to catch mechanical issues using simulated fault data. The primary success criterion is accuracy: the system must correctly identify emerging wear and tear while strictly minimizing "false alarms" (warning the driver when nothing is actually wrong). We will also evaluate how much earlier our system detects a problem compared to standard warning lights.
- **UI Performance:** The Heads-Up Display software will be evaluated on its visual stability. The interface must run smoothly on the Raspberry Pi without any screen freezing, crashing, or lagging, ensuring the driver is never distracted by a glitchy display.

11 Cost Analysis

Most of our costs are associated with hardware components as all the software we would need is free or can be done for free. This helps bring our cost to roughly \$418 and leaving us with \$282 in case we run into unexpected costs.

Table 3: Estimated project budget.

Item	Unit cost	Qty	Total
DB9 Cable	\$10	1	\$10
OBD II-UART-v13 (Raspberry Pi)	\$63	1	\$63
12V to 5V buck converter	\$10	1	\$10
3D Printed Enclosure and mounting	Free	1	Free
Raspberry Pi 4/5 8GB	\$180	1	\$180
TI DLPCLCR2000EVM	\$100	1	\$100
Front surface mirror	\$15	1	\$15
Beamsplitter glass	\$40	1	\$40
Total			\$418

12 Team Contribution

The team is split into two focus areas. One is software and AI work (decoding logic, data pipeline, the predictive model, and the HUD application), while the other is hardware work (OBD-II interfacing, the optical/projection path, power, and physical integration). All members can shift between the two sides as the workload demands, ensuring whichever subsystem is on the critical path has enough hands. Additionally, all members contribute to system integration, testing, and the final report and presentation.

Keaton Zelazny — AI / Machine Learning.

1. Design, train, and tune the LSTM autoencoder for anomaly detection using TensorFlow, and export it to TensorFlow Lite for on-device inference.

2. Implement the rolling z-score baseline and run the comparative evaluation (reconstruction error, precision/recall, false-positive rate, detection lead time).
3. Build the synthetic anomaly-injection routines and manage the train/test split across the three data sources.

Klaidas Wikar — Software / Decoding.

1. Develop the CAN/ISO decoding logic and the JSON configuration file that maps raw trouble codes to plain-English descriptions.
2. Build the data pipeline: poll the OBD-II stream via python-obd, clean and timestamp PID readings, and log them to SQLite/CSV for the model.
3. Integrate the trained model's output into the live application as a structured health/anomaly score.

Siwon Kang — Display Software & AI.

1. Build the full-screen PyGame HUD: customizable gauges, the ML-driven status overview, and critical error-code alerts (including horizontal mirroring for the combiner glass).
2. Implement the stale-data handling (last-known-good readings with a visual staleness indicator).
3. Support model deployment and inference on the Raspberry Pi.

Neil Singh — Hardware / Integration.

1. Set up and validate the OBDII-UART-v13 board (STN1110) connection to the vehicle's OBD-II port.
2. Establish and stabilize the Bluetooth link between the UART board and the Raspberry Pi.
3. Own end-to-end hardware–software integration, wiring the full pipeline (OBD → decode → Pi → display).

Ajey Sasimugunthan — Optical / HUD Hardware.

1. Configure the TI DLP Pico projector and tune the projection output for the HUD.
2. Build and align the optical path (fold mirror → beamsplitter/combiner glass) for a clear, readable reflected image.
3. Design and 3D-print the enclosure that houses the projector–mirror assembly and electronics.

Noah Carlson — Power & Hardware.

1. Implement and verify the 12 V → 5 V/3.3 V buck-converter power regulation for all modules from the vehicle's outlet.
2. Handle physical mounting and lead the collection of self-recorded baseline driving data from the team's test vehicle.
3. Conduct hardware bring-up and verification testing (continuity, voltage levels, and thermal checks) to confirm each module is safe and stable before integration.

13 Proposed Timeline

The schedule below covers the build semester (January–April), which is the primary development period. Planning and logistics—finalizing the bill of materials, ordering hardware, sourcing OBD-II datasets, and locking in the AI approach—are completed in the preceding fall term so that development can begin immediately. The plan front-loads a minimal end-to-end pipeline and then advances the hardware, software, and AI subsystems in parallel before final integration and in-vehicle testing.

Table 4: Milestone schedule by week (build semester).

Weeks	Milestones
1–2	Flash Raspberry Pi OS Lite and set up the dev environment; establish the OBD-II connection and log raw data from a test vehicle.
3–5	Build the CAN/ISO decoding logic and JSON config; turn raw codes into clean, structured data.
5–8	Prototype the PyGame HUD (gauges + error-code panel); begin bench tests of the projector → mirror → combiner optical path.
6–11	Train and validate the LSTM autoencoder against the rolling z-score baseline; tune thresholds on the synthetic anomaly set.
9–11	3D-print the enclosure, mount components, and finalize the 12 V → 5 V / 3.3 V power regulation.
10–12	Wire the full pipeline end-to-end and integrate the model into the live data stream; first full-system bench test running off vehicle power.
13–15	In-vehicle testing under real driving conditions; tune AI accuracy (false positives/negatives) and polish the UI.
16	Buffer for debugging and final tweaks; complete the final report, poster, and demo/presentation.

14 Next Semester Plan

With the planning phase complete, next semester is dedicated to building, integrating, and validating a working Break Check prototype end-to-end.

Goals. Deliver a functioning prototype that reads live OBD-II data, decodes it into plain-English information, runs the predictive model on-device, and projects a clear HUD into the driver’s line of sight—meeting the requirements in Table 1 (sub-second display latency, stable 30-minute Bluetooth link, and $\geq 90\%$ model accuracy).

Planned Work. Stand up the OBD-II → UART → Pi data path and decoding logic; collect 10–15 hours of baseline driving data and assemble the full training corpus; train and quantize the LSTM autoencoder for TensorFlow Lite; build the PyGame HUD; assemble the optical/projection hardware and enclosure; and integrate all subsystems on vehicle power.

Milestones and Deliverables. Working OBD-II data capture (Weeks 1–2); structured decoded data (Week 5); demoable HUD and a validated model (Weeks 8–11); first full-system bench test (Week 12); successful in-vehicle test (Week 15); and the final prototype, report, poster, and demo (Week 16).

Anticipated Challenges. Optical alignment of the mirror and combiner glass is expected to be more time-consuming than planned, so it begins early as a parallel track. Model quality depends on representative data, which is why dataset collection and synthetic anomaly injection start up front. To manage these risks, the rolling z-score baseline serves as a fallback if the autoencoder does not yield a meaningful improvement in detection lead time, and the final two weeks are reserved as buffer for debugging and integration issues.

15 Conclusion

Dashboard warning lights only tell you when something is already broken. Break Check changes this by catching issues early.

We are building a smart, transparent HUD using a Raspberry Pi, an OBD-II translator, and a predictive AI model. Instead of just scanning for standard error codes, our system learns how the car normally drives. This allows us to flag subtle mechanical drifts before they become expensive repairs, and project those warnings safely into the driver's line of sight.

By relying on standard hardware and open-source libraries, our estimated budget is strictly capped at \$418. This makes the project highly feasible for the capstone timeline. Ultimately, we will deliver a working prototype that helps drivers understand their vehicle's health, preventing severe damage and cutting down unnecessary mechanic visits.

References

- [1] E. Pena and M. G. Legaspi, "UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter | Analog Devices," *Analog.com*, 2024. <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>
- [2] C. Velasquez, "Analyzing Rolling Z-Score in Stock Trading with Python," *Entrepreneurdly*, Feb. 12, 2024. <https://entrepreneurdly.com/analyzing-rolling-z-scores-in-stock-trading-with-python/>. [Accessed: Jun. 16, 2026].
- [3] S. Buschjäger, P. -J. Honysz and K. Morik, "Generalized Isolation Forest: Some Theory and More Applications Extended Abstract," 2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA), Sydney, NSW, Australia, 2020, pp. 793-794, doi: 10.1109/DSAA49011.2020.00120. keywords: Forestry;Vegetation;Anomaly detection;Clustering algorithms;Runtime;Machine learning;Knowledge discovery;Outlier Detection;Isolation Forest;Density Estimation;Ensemble;Tree,
- [4] D. Nikitha, D. R. R. S. Kalaiarasi and R. Durga, "Improved Accuracy in Detection of Driver Drowsiness Using Novel Random Forest Classifier Compared with Decision Tree Classifier," 2024 IEEE 9th International Conference on Engineering Technologies and Applied Sciences (ICETAS), Bahrain, Bahrain, 2024, pp. 1-5, doi: 10.1109/ICETAS62372.2024.11119842. keywords: Accuracy;Sleep;Wheels;Sensor systems;Road safety;Sensors;Decision trees;Random forests;Vehicles;Classification tree analysis;Driver Drowsiness detection;Electrocardiogram;Sensors;Novel Random Forest Classifier;Machine Learning;Decision Tree Classifier,
- [5] K. Lim and S. Prum, "Machine Learning Approaches for Anomaly Detection in IoT Networks: A Survey," 2025 International Conference on Software, Knowledge, Information Management Applications (SKIMA), Paisley, United Kingdom, 2025, pp. 1-6, doi: 10.1109/SKIMA66621.2025.11155642. keywords: IoT Security;Anomaly Detection;Machine Learning;Deep Learning;Federated Learning;Intrusion Detection;IoT Networks,
- [6] C. Barreto, "OBD-II datasets," *Kaggle.com*, 2018. <https://www.kaggle.com/datasets/cephasax/obdiids3>

A Appendix: Additional Material

[Detailed tables, extended results, code listings, or a detailed system diagram.]

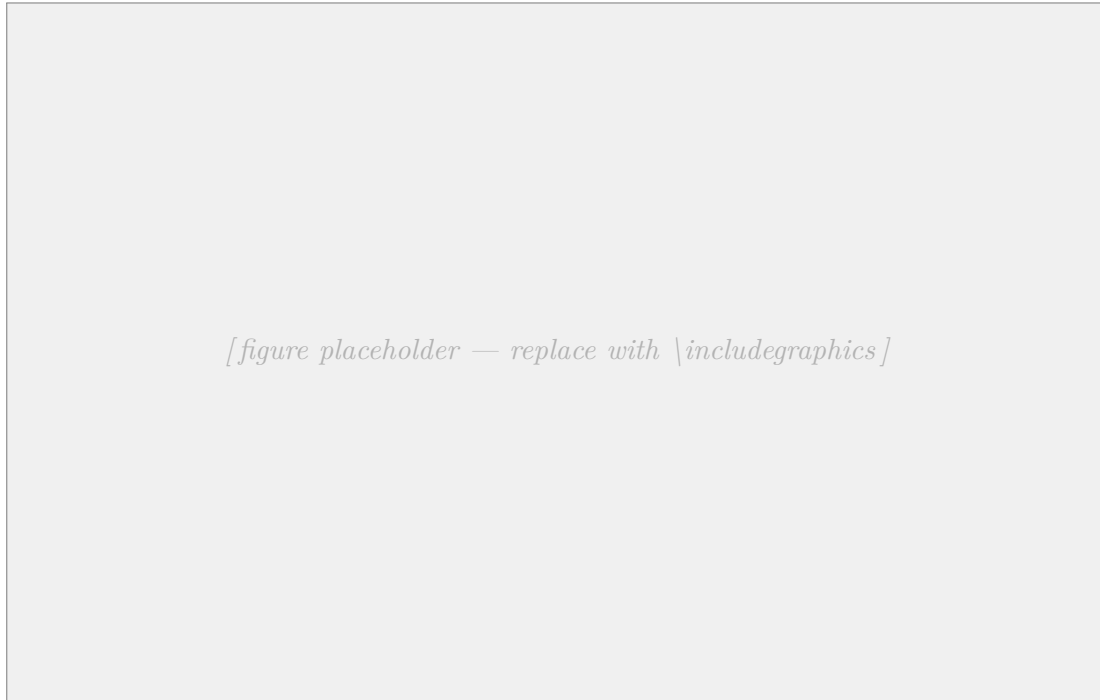


Figure 3: Detailed system / workflow diagram.